Test and Practice 2 - Threads and Memories

For this TP, a **report** of txt format is to be written and uploaded with the codes. Codes should be clean, concise and commented. Avoid anything unrelated to the computation. All the necessary files can be downloaded from e-learning. The CUDA runtime API reference (the doc) is here.

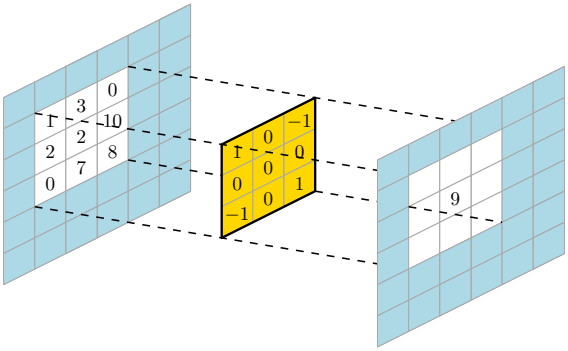
Exercise 1: Time your kernel
Complete the code in `cuStopwatch.cu`, which is a class of timing a kernel using CUDA events. You can look up `cudaEventRecord` and `cudaEventElapsedTime` in the doc.

Exercise 2: Someone is not working
Use `cuStopwatch` to time the pairs of kernel invocations in `starve1.cu`, `starve2.cu`, `starve3.cu` and `starve4.cu`. Why is there a difference in each pair? Explain.

Problem 1: Convolutive filters for images
In image processing, *convolutive filters* are widely used. We now implement 3×3 convolutive filters using CUDA. A grayscale image can be viewed as a matrix $M = (m_{i,j})$ of pixels, where $m_{i,j}$ stands for the luminosity of the pixel at column i and row j. A 3×3 convolutive filter is a 3×3 matrix $K = (k_{u,v})$. Using M as input image, the output image $M' = (m'_{i,j})$ is given by

$$m'_{i,j} = \sum_{u=1}^{3} \sum_{v=1}^{3} k_{u,v} m_{i-2+u,j-2+v}.$$

Informally speaking, to compute $m'_{i,j}$, we overlay the filter K on the input image M such that the center of K is at column i and row j of M. We then multiply overlapping entries of K and M, then sum up the products. The result is taken as $m'_{i,j}$. Here is an illustration:



On color images with pixels represented by the luminosity of red, green and blue, we perform the filter on each color, and combine them for the output image. Upon invalid index access on images, we take the *clamp to edge* convention, where we use the valid pixel closest to the index access.

In the file `tp2-problem.zip`, there is the source code `conv-filter.cu` of our program. After compilation, the program takes several arguments:

- First argument: path to the input BMP. You can test with `cat.bmp` in the files.
- Second argument (optional): the filter to apply (`identity`, `edge`, `sharp`, `gauss`).
- Thirt argument (optional): `texture` if texture memory is used for the image.

Once the functions are completed, you can compile the source with `compile.cmd`. Input and output images are represented as a 1D array `unsigned char* pixels`, in which each pixel is represented by 3 `unsigned chars` of its red, green and blue components, and pixels are listed row after row.

1. Explain why we choose to store the convolutive filter in constant memory.
2. Complete the device kernel `conv_global`. It should compute on each thread one pixel of the output image. Input image should be placed in global memory.
3. Write the host function `conv_global_gpu` that allocates memory space, invokes `conv_global` and frees the allocated space. It should return the time taken for kernel execution. Use your class `cuStopwatch` to do the timing.
4. **(Bonus)** Complete the device kernel `conv_texture`, which performs exactly the same task as `conv_global`, but using texture memory for the input image.
5. **(Bonus)** Write the host function `conv_texture_gpu` as the counterpart of `conv_global_gpu` for `conv_texture`. For allocation of texture, please adapt the code in CUDA C++ Programming Guide.
6. **(Bonus)** Observe the timing difference. Try to search online to give an explanation.
7. **(Bonus, hard)** Write a kernel using shared memory for acceleration.